



B TREE AND B+ TREE

ALEYNA AĞIRMAN

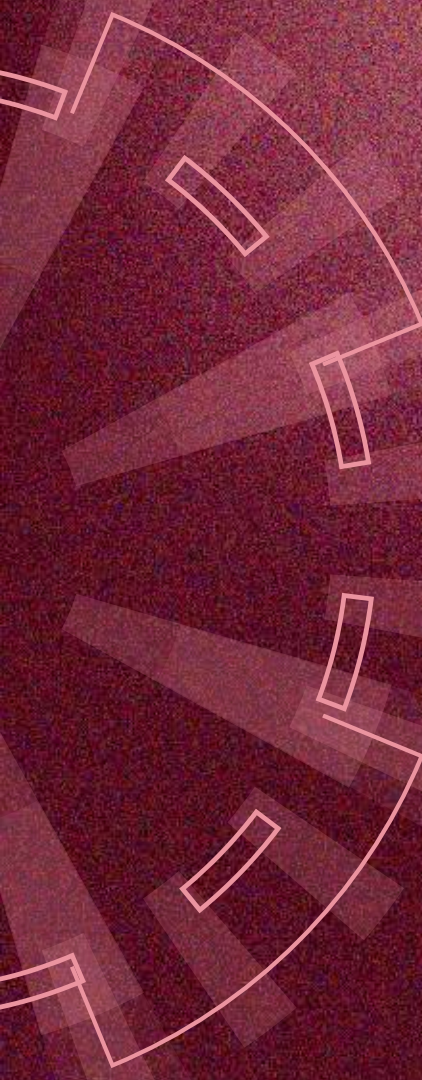
B TREE VE B+ TREE NEDİR

B-Tree (Balanced Tree)

- Veri yapılarında kullanılan dengelenmiş çoklu arama ağacıdır.
- Her düğümde çok sayıda anahtar tutabilir.
- Yapı yüksekliği mümkün olduğunca kısa tutulur → disk erişimi azalır.
- Yapraklar aynı seviyededir → arama işlemlerinde tutarlılık sağlar.

B+ Tree

- B-Tree'nin geliştirilmiş halidir.
- Tüm gerçek veriler yaprak düğümlerde tutulur, iç düğümler yalnızca yönlendirme (index) içerir.
- Yaprak düğümler birbirine bağlıdır → sıralı veri taramaları çok hızlıdır.



Günümüzde Kullanım Alanları

Veri Tabanlarında Kullanımı

Modern veritabanları indeksleme için yoğun bir şekilde B+ Tree kullanır.

Neden?

- Disk I/O maliyeti çok düşüktür.
- Sorgular için logaritmik arama sağlar.
- Sıralı veri taraması gereken işlemler için range query performansı çok yüksektir.

Örnek Sistemler

- MySQL (InnoDB → Varsayılan indeks yapısı B+ Tree)
 - PRIMARY KEY → B+ Tree
 - Secondary Index → B+ Tree
 - PostgreSQLB-Tree → varsayılan indeks
 - Çoklu veri türlerini destekleyen optimize edilmiş uygulama
 - MongoDBTüm index yapıları B-Tree varyantıdır (genellikle B+ Tree yaklaşımı)

Arama Motorları ve Büyük Veri Sistemleri

Neden kullanılır?

- Büyük veri ortamlarında disk tabanlı arama çok kritiktir.
- B+ Tree yapısı seri okuma (sequential scan) için uygundur.
- Dağıtık sistemlerde indeks tutarlılığı sağlar.

Kullanan sistem örnekleri

- Elasticsearch → B-Tree tabanlı segment indeksleri
- Hadoop HBase → LSM-Tree + B-Tree hibrit yapılar
- Spark SQL → Parquet dosyalarında B-Tree benzeri sayfa indeksleri
- Google arama altyapısında kullanılan eski dosya sistemlerinin çoğunda B-Tree tabanlı meta veri yapıları bulunur.

Dosya Sistemlerinde Kullanımı

Kullanan dosya sistemleri

- NTFS (Windows) → Master File Table (MFT) B-Tree tabanlı
- HFS+ (macOS) → tamamen B-Tree tabanlı katalog kayıtları
- EXT4 (Linux) → dizin yapısı B-Tree benzeri HTree kullanır

Neden kullanılmaktadır?

- Bir dizindeki binlerce dosyayı çok hızlı listeleyebilmek için
- Dosya adlarına göre sıralı tarama yapabilmek için
- Arama işlemlerinin $O(\log n)$ olmasını sağlamak için

Neden Modern Sistemlerde Tercih Edilmektedir?



1. Disk-Tabanlı Sistemlere Uygunluk
 - Disk erişimi RAM'den milyonlarca kat daha yavaştır.
 - B+ Tree az sayıda disk erişimi ile sonuca ulaşır.
2. Sıralı Veri İşlemede Üstünlük
 - Range query işlemlerinde (ör. "1000 – 2000 arasındaki ürünleri bul")
 - Yaprakların bağlı olması → çok hızlı tarama sağlar.
3. Çok Büyük Veri Miktarlarında Stabil Performans
 - veri arttıkça performans bozulmaz
 - Yüksek dallanma faktörü sayesinde ağaç yüksekliği küçüktür
4. Eş zamanlı işlemlere uygunluk
 - Kilitleme (locking) ve eş zamanlı güncellemeler için güvenlidir

Günlük Hayattan Özgün Benzetmeler

Çok Çekmeceli Döküman Dolabı"

- Bir ofiste düşün: Büyük bir metal belge dolabı var.
- Her çekmecedeki birden fazla klasör bulunuyor.
- Çekmeceler alfabetik sırada dizilmiş.

Benzetme:

- Çekmece → B-Tree düğümü
- İçindeki klasörler → anahtarlar
- Alt çekmecelere referans → pointer'lar
- En alttaki çekmeceler → yaprak düğümler

Aradığın belgeyi bulmak için:

1. Dolabın üzerindeki etiketlere bakarsın (üst düğüm)
2. Harf aralığını bulursun (anahtar karşılaştırması)
3. İlgili çekmeceye inersin (pointer takibi)
4. Belgeyi bulursun (yaprak düğüm)



Böyle Bir Veri Tabanında Bu Ağaç Yapılarının Somut Faydası

Çok Hızlı Arama

Örneğin bir hastane otomasyonunda:

- Hastanın T.C. kimlik numarasına göre arama → B+ Tree ile $O(\log n)$
- Sıralı hasta listesi çıkarma çok hızlı olur.

Disk Operasyonlarını Azaltma

- Bir sorgu çalıştığında diskten sayfa sayfa veri okunur.
- B+ Tree yüksek dallanma sayesinde az sayıda sayfa okutur.

Sıralı Raporlamayı Hızlandırma

- “Mart ayında yapılan tüm işlemler” gibi sorgular yapraklar arası hızlı geçiş sayesinde saniyeler içinde çıkar.

Büyük veri setlerinde ölçeklenebilirlik

- Veri milyonlara, milyarlaraya çıksa bile performans sabittir.

Bu Veri Yapıları Neden Önemlidir



Temel nedenler

- Veriyi dengeli tutarlar
- Arama, ekleme, silme her zaman $O(\log n)$
- Disk tabanlı sistemler için optimize edilmişlerdir
- Sıralı veri taramasında çok hızlıdırlar
- Büyük veri ortamları için olmazsa olmazdırlar

Büyük Veri Çağında Neden İhtiyaç Duyulmaktadır



Veri Boyutunun Devasa Hale Gelmesi

- 1 milyar kayıt içeren sistemlerde klasik binary tree çöker.
- B+ Tree sabit performans sağlar.

Sürekli Artan Depolama Gereksinimi

- Disk erişimi pahalıdır → B+ Tree minimize eder.

Dağıtık sistemlerde indeks yönetimi

- NoSQL, Hadoop, Spark gibi sistemlerin temel yapı taşlarında B-Tree mantığı vardır.



Gelecekte de Kullanılmaya Devam Edecek mi?

KESİNLİKLE EVET!!!



Sepepler:

- Disk temelli veri yapıları hâlâ en ekonomik çözüm
- SSD'ler artsa bile rastgele erişim hâlâ pahalı
- Yeni yapıların çoğu (LSM-Tree vs.) bile B+ Tree mantığını temel alıyor
- Veri büyüdükçe "sıralı erişim" önemi artmaya devam edecek
- Endüstri standardı olarak yerleşmiş durumda

Gelecekte olası değişim:

- Bellek tabanlı veri tabanlarında hibrit yapıların artması
- Ancak disk tabanlı sistemler var oldukça B+ Tree kullanılacak



TEŞEKKÜRLER...